

1. strlen() :-

strlen() returns the no. of characters in a string used excluding to find the length of a string. the null terminator \0.

Syntax

C

```
size_t strlen (const char *str);
```

Example :-

```
#include <stdio.h>
#include <conio.h>
#include <string.h>
```

```

int main () {
    char s[] = "Hello";
    int len = strlen(s);
    printf("length = %d", len);
    // output : 5
    return 0;
}

```

2. strcpy () :-

strcpy () copies the contents of one string (source) into another string (destination), including the terminating '\0'.

Syntax:-

```
char *strcpy(char *dest, const char *src);
```

Example:-

```

#include <stdio.h>
#include <string.h>

int main () {
    char src[] = "Hello";
    char dest[20];
    strcpy(dest, src);
    printf("source : %s \n", src);
    printf("Destination : %s \n", dest);
}

```

```
return 0;
```

```
}
```

Output :-

Source : Hello.

Destination : Hello.

3. strncpy() :-

strncpy() copies at most n characters from the source string a copy fixed number of characters from one string to another.

Syntax:-

```
char * strncpy (char *dest, const char *src, size_t n)
```

Example:-

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main () {
```

```
char src[] = "cat";
```

```
char dest[10];
```

```
strncpy (dest, src, 10);
```

```
// copy up to 10 characters.
```

```
printf("dest = %s", dest);
```

```
return 0;
```

```
}
```

output:-

dest = cat.

4. strcat():-

strcat(destination, source) This function appends the source string to the end of the destination string.

Header file:-

#include <string.h>

Syntax

strcat(char destination[], const char source[]);

Example:-

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
int main () {
```

```
    char str1[20] = "Hello";
```

```
    char str2[] = "World";
```

```
    strcat(str1, str2);
```

```
    printf("concatenated string: %s", str1);
```

```
    return 0;
```

```
}
```

Output:-

concatenated string: Hello world.

5. strncat() :-

strncat(destination, source, n) appends first n characters of source to the end of destination.

Syntax

```
strncat(char destination[], const char source[], size_t n);
```

Example:-

```
#include <stdio.h>
#include <string.h>

int main () {
```

```
    char str1[20] = "Hello";
```

```
    char str2[] = "World !!!";
```

```
    strncat(str1, str2, 5);
```

// Append only first 5 characters of str2.

```
    printf("Result: %s", str1);
```

```
    return 0;
```

```
}
```

Output:-

Result : Hello woorld.

6. strcmp() :-

strcmp(str1, str2) compares two strings lexicographically (dictionary order).

Header file:-

```
#include <string.h>
```

Syntax

```
int strcmp (const char* str1, const char* str2);
```

Example :- #include <stdio.h>

```
#include <conio.h>
```

```
int main() {
```

```
char s1[] = "Apple";
```

```
char s2[] = "Banana";
```

```
int result = strcmp(s1, s2);
```

```
if (result == 0)
```

```
printf("Both strings are equal");
```

```
else if (result > 0)
```

```
printf("s1 is greater than s2");
```

```
else
```

```
printf("s1 is smaller than s2");
```

```
return 0;
```

```
}
```

Output :-

s1 is smaller than s2.

7. Strncmp() :-

```
strncmp(str1, str2, n).
```

Compares at most n characters of str1 and str2.

Syntax

```
int strncmp (const char* str1, const char* str2,  
size_t n);
```

Example:-

```
#include <stdio.h>
#include <conio.h>

int main () {
    char s1[] = "Application";
    char s2[] = "Apple";
    int result = strcmp(s1, s2, 5);
    // compare first 5 characters.
    if (result == 0)
        printf("First 5 characters are equal");
    else
        printf("First 5 characters are NOT equal");
    return 0;
}
```

Output:-

First 5 characters are equal.